Faculty: Computers and Informatics, Zag. Uni **Class Year:** 2024/2025 **Year:** Level two - General **Subject Name:** Data Structures **Code:** CS200	**Mid-term Exam**	**Date:** Nov 23. 2024 **Time Allowed:** 1 Hour. **No. of Questions:** 3 **Degree:** 15 **Number of pages:** 2

4 **Question 1: - Fill the following space**

1. Data structure in which data items are arranged in non-sequential order (hierarchical manner) is called .....
2. The elements in ..... data structure may or may not be the same such as structures.
3. ..... is the way we look at a data structure, focusing on what it does and ignoring how it does its job.
4. ..... is a design principle in C++ where resources like dynamic memory are acquired and released automatically through constructors and destructors.
5. A ..... is a linear collection of elements that follows a FIFO order
6. The ..... operation retrieves the top element of the stack without removing it.
7. The limitation of simple queue is ..... while
8. One of the physical data structures is ..... is an example of logical data structures.

5 **Question 2: - Compare the following**

	Static Array	Dynamic Array
Memory Management		
Memory Location		
Access Speed		

	Simple Queue	Circular Queue
Type of Increment		
What is the code's statement of rear increment to make enqueue operation		
What is the condition of IsFull operation using rear & front		

6 **Q3: - Complete the following functions to:**

1. Remove all the duplicated elements from Duparr.

```
int Duparr[] = { 1, 2, 2, 3, 3, 4, 5, 5 };
int removeDuplicates() {                    // T(N) Must be O(N)



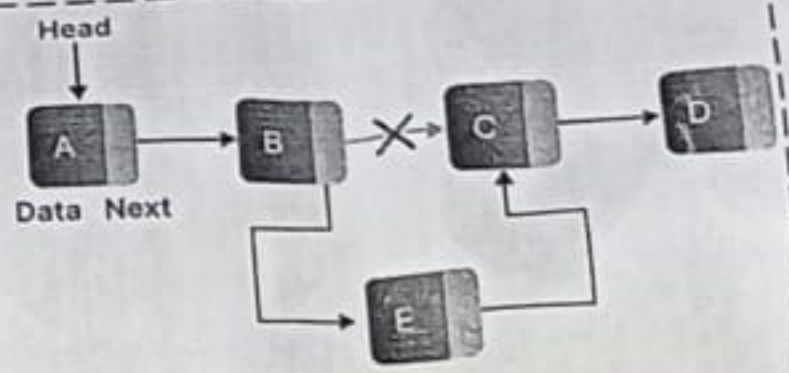

}
//Output: {1,2,3,4,5} with newsize=5
```

2. Insert an element at a specific position on a singly linked list.

```
void InsertataGivenPosition(int value, int position)
{
        // Memory Allocation
        Node* newNode = _____________________________
        if (position == 1) {
                _____________________________
                _____________________________

        }
        else {
                Node* temp = head;
                for (int i = 1; i < _________ && temp != _________;i++) {
                        temp = temp->next;
                }
                if (temp == nullptr) {
                        std::cout << "Position out of bounds" << std::endl;
                }
                else {
                        _____________________________
                        _____________________________

                }
        }
}
```

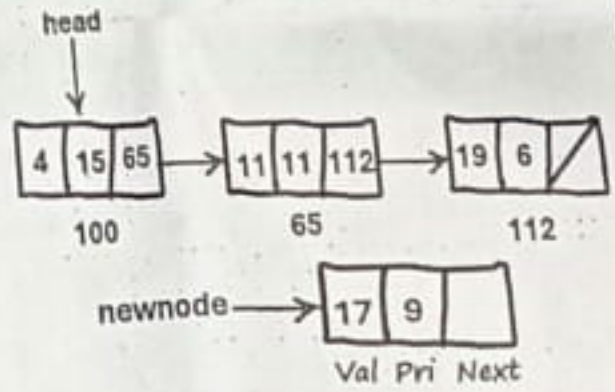


3. Insert an element in a priority queue (sorted insertion)

```
void enqueue(T Val, int Pri) {
// 1. Memory allocation for the new node
_____________________________

// 2. Traverse list to find the correct position to insert
Node<T>* temp = head;
while (_____________________________)
{
        temp = temp->next;
}
// 3. Update Pointers
        _____________________________
        _____________________________

}
```



Good Luck ^_^